Timing-Safe and Latency-Insensitive Hardware Design Methodologies

An Annotated Bibliography

Ruijie Gao
ruijieg@umich.edu

April 20, 2026

Introduction

Hardware design today resembles software before type systems. A Verilog module header such as `module mul(input [31:0] a, b; output [31:0] out)` says how wide the wires are, but not "give me operands on cycle 0 and I will return the product on cycle 3." That contract lives in a datasheet or in an engineer who left three years ago; call the module at the wrong cycle and it silently returns garbage that the synthesizer compiles into a $10M chip. PL readers will recognize each pain point as a problem already solved for software: "when is this value valid?" (lifetimes, Rust); "can I call this module now?" (typestate, session types); "does composition preserve the contract?" (linearity); "does behaviour depend on invisible scheduling?" (data races).

A *hardware description language (HDL)* – Verilog, VHDL, or a modern relative – is the source language of a chip; a *synthesizer* lowers it to a gate-level netlist, much as a C compiler lowers to assembly. Most HDL is written at the *register-transfer level (RTL)*: each line describes a *wire* (pure combinational logic) or a *register* (state advanced by a global clock ticking in lockstep, so "cycle N" is just the N-th tick). What RTL omits is *when* each wire is meaningful.

Below 100 nm feature sizes (the *deep-submicron* regime), a signal crossing a chip needs several cycles to settle, breaking the synchronous assumption that every wire arrives within one tick [1]. Modern systems-on-chip compose hundreds of reusable *IP (intellectual-property) blocks* from dozens of vendors, and no engineer holds the whole picture. The community has responded along two PL-inspired lines. *Latency-insensitive design* replaces "arrives by cycle N" with a valid/ready *handshake* on every interface – `async`/`await` for circuits, where either side may stall. *Timing-safe HDLs* go further, lifting the hidden timing contract into the module's type signature so a type checker can reject ill-timed programs before synthesis.

Summary of Key Works

The theoretical foundation is Carloni, McMillan, and Sangiovanni-Vincentelli (2001) [1], who define a *patient process* as a circuit that can be stalled at any moment and resumed without corrupting state. They prove a compositional *latency-equivalence* theorem: replacing every strict process in a synchronous system with its patient counterpart preserves the observable output stream regardless of per-channel delay. A *relay station* – a one-cycle patient buffer – can therefore be inserted on any wire that fails timing closure without changing functional behaviour. In PL terms, this is a bisimulation: observable streams agree even as internal schedules diverge.

A decade later, Lockhart, Zibrat, and Batten's PyMTL (2014) [2] turned LI interfaces into a practical composition primitive. Architects routinely iterate through three modelling abstractions – a *functional-level* (FL) model ("is the algorithm right?"), a *cycle-level* (CL) model ("how many cycles?"), and an RTL model ("can we synthesize it?") – historically written in different languages, producing a "methodology gap." PyMTL is an embedded DSL in Python that supports all three levels and requires every module boundary to use a valid/ready handshake, so FL, CL, and RTL components freely compose. A companion JIT specializer (SimJIT) brings Python simulation within 4–6× of hand-written C++ and Verilator-compiled Verilog on a dot-product accelerator and a 64-node mesh network. PyMTL turns Carloni's interfaces from a theoretical device into a cross-abstraction programming discipline.

Pan and Batten (2023) address a verification gap [5]: even when every module honours valid/ready, its *internal* logic may hide bugs triggered only by specific stall sequences, whose number grows exponentially in pipeline depth. They formalize the *stall invariant property* – the sequence of data-transferring cycles on a module's egress must be identical under any two stall patterns – and propose *Bounded Latency Equivalence Checking (BLEC)*: instantiate two copies of the module with different stall patterns and have a commercial formal tool prove their egress streams equivalent up to a bounded depth. On three case studies (an LI processing element, a GCD unit, and a pipelined RISC-V processor), BLEC detects bugs, such as a pipeline-register enable that forgets to observe a downstream stall, that simulation misses.

Can we reject ill-timed designs at compile time instead? Nigam, Azevedo de Amorim, and Sampson (2023) answer yes with *Filament* [3], an HDL with *timeline types*. A signature `Mult<T: 3>(@[T, T+1] l, r: 32) -> (@[T+2, T+3] out: 32)` reads like a refinement type over a cycle index: an event variable T; a three-cycle *initiation interval*; inputs valid on $[T, T+1]$; output on $[T+2, T+3]$. The type system, inspired by separation logic, treats each instance's timeline as a resource, and a soundness theorem proves that well-typed programs compile to circuits free of structural hazards. Filament types 14 designs from Aetherling (a streaming DSL), uncovering five latency bugs in Aetherling's output, and a Filament/Reticle `conv2d` uses 7× fewer logic cells than Aetherling's own backend.

Filament's types are fixed at design time, but real hardware uses *parameterized generators* (e.g., FloPoCo floating-point or Vivado IP) whose latency depends on inputs like bitwidth. HDL parameters conventionally flow only parent-to-child, so a generated child cannot propagate "I will have latency 4" back upward. Nigam et al.'s Lilac (ASPLOS 2026) [4] extends Filament with *output parameters*: compile-time values emitted by a child and consumed symbolically by its parent, with an SMT solver discharging hazard-freedom for every parameterization. On a FloPoCo FPU, a Lilac *latency-abstract* design uses 26–33% fewer LUTs and registers and runs at 6.8% higher frequency than the same modules wrapped in an LI interface – quantifying the cost of the handshake logic Carloni's theory imposes.

Timeline types assume every latency is a compile-time constant, but caches, page-table walkers, and bus protocols have *data-dependent* timing by design. Yu et al.'s Anvil (ASPLOS 2026) [6] generalizes Filament-style safety by indexing time with *events* – named points in a per-module event graph – rather than integer cycles. A channel `right req: (addr @ req->res)` reads "the address is valid from `req` until the next `res`"; the compiler rejects any use outside that lifetime, analogous to Rust lifetimes lifted to runtime events. On ten designs Filament cannot express (the CVA6 RISC-V TLB and page-table walker, OpenTitan's AES, AXI-Lite routers, and common FIFO cells), Anvil's SystemVerilog averages 4.5% area and 3.75% power overhead with zero extra cycle latency.

Conclusion and Open Questions

Across two decades the field has moved hardware's hidden timing contract out of the datasheet and into the type system: Carloni made LI composition provably safe; PyMTL turned it into a cross-abstraction programming discipline; BLEC gave us formal verification of the resulting RTL; Filament and Lilac made timing contracts static and increasingly expressive; Anvil made them dynamic and general-purpose. Several open questions remain. (1) Can one language combine Filament's zero-overhead static checking on deterministic paths with Anvil's event-based dynamic checking elsewhere, so each region of a design pays only for the discipline it needs? (2) When a timeline type proves safety internally, can the compiler *remove* the valid/ready FSM whose area cost Lilac measures and emit LI wrappers only where a static schedule is insufficient? (3) Rather than type-check hand-written controllers, can we *synthesize* correct-by-construction LI controllers from temporal-logic specifications (LTL, GR(1)) and validate them with BLEC-style equivalence? (4) Can bounded equivalence checking scale to industrial SoCs via compositional assume/guarantee reasoning along the very LI interfaces these papers formalize?

Updated Implementation Proposal

My implementation explores the third open question: reactive synthesis of latency-insensitive controllers from temporal-logic specifications. I will build a tool that takes a specification of an elastic-buffer or pipeline controller's valid/ready behaviour, written in GR(1) (a tractable fragment of linear temporal logic), and produces synthesizable Verilog whose correctness is guaranteed by the synthesis procedure. The specification-to-automaton step will use Spot's `ltlsynt` backend to produce a Mealy machine; a second pass will lower the automaton into RTL.

I will evaluate the approach on three designs of increasing complexity: (1) a single relay station with forward backpressure, (2) a two-slot elastic buffer, and (3) a multi-stage pipeline controller with stall propagation. Each synthesized controller will be compared against a hand-written baseline along three axes: *correctness*, via BLEC-style stall-invariant equivalence checking on the resulting RTL [5]; *area and frequency*, via Vivado synthesis targeting an FPGA; and *specification effort*, measured in lines of GR(1) versus lines of Verilog. A negative result (for example, a synthesized FSM that is unacceptably large) is itself an interesting data point about the current state of reactive synthesis for hardware.

References

[1] L. P. Carloni, K. L. McMillan, and A. L. Sangiovanni-Vincentelli, "Theory of latency-insensitive design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 10, pp. 1059–1076, 2001.

> Specific problem: as chip feature sizes shrink below 100 nm ("deep submicron"), a wire crossing the chip takes more than one clock cycle to settle, invalidating the traditional synchronous assumption that every signal reaches every register within a single tick. Recovering functional correctness then requires costly iterations between logic synthesis and physical placement. Solution: the authors define a *patient process* as a circuit that can be stalled at any moment—via a valid/stall signalling protocol—and resumed without corrupting state, and they insert *relay stations* (one-cycle patient buffers) on long wires to segment them into timing-closable pieces. They prove a compositional *latency-equivalence theorem*: if every strict process in a synchronous

system is replaced by its patient counterpart, the composed system emits the same ordered stream of observable outputs regardless of per-channel delays. Evaluation is a mixture of paper proofs of latency equivalence and a case-study redesign of the PDLX out-of-order microprocessor, showing that relay stations can be inserted mechanically without changing observable behaviour. Limitations / future work: the theory assumes fully synchronous, deterministic processes and does not address asynchronous or data-dependent (dynamic) timing. The paper acknowledges but does not quantify the area cost of the valid/ready/stall logic and relay registers; later work (Lilac) measures it as roughly 26–33% in LUTs and registers for parameterized pipelines.

[2] D. Lockhart, G. Zibrat, and C. Batten, "PyMTL: A unified framework for vertically integrated computer architecture research," in *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2014, pp. 280–292.

Specific problem: computer architects iterate through three modelling abstractions—functional-level ("is the algorithm right?"), cycle-level ("how many cycles?"), and register-transfer-level ("can we synthesize it?")—that are conventionally written in three different languages (Python/MATLAB, C++, Verilog/VHDL) with incompatible concurrency models, producing a *methodology gap* that slows design-space exploration. Solution: PyMTL is an embedded hardware-modelling DSL hosted in Python that supports all three abstractions in one language and enforces *latency-insensitive* valid/ready interfaces at every module boundary, so that models at different levels of detail can be freely composed and co-simulated. A companion contribution, SimJIT, is a selective just-in-time specializer that translates PyMTL cycle-level and RTL models into optimized C++; the framework also includes a translator that emits synthesizable Verilog from the subset of PyMTL used for RTL. Evaluation uses two benchmarks: a dot-product accelerator tile (modelled at varying levels of detail from pure FL to pure RTL) and a 64-node mesh network-on-chip. SimJIT combined with the PyPy tracing JIT delivers up to 72× speedup for CL models and 200× for RTL models over CPython, bringing Python simulation within 4–6× of hand-written C++ and Verilator-compiled Verilog respectively. Limitations / future work: only a restricted translatable subset of PyMTL RTL actually reaches Verilog; FL and CL models remain simulation-only. Latency-insensitive interfaces at every boundary give modularity but impose handshake overhead that the later timing-safe HDL line of work (Filament, Lilac, Anvil) aims to reduce.

[3] R. Nigam, P. H. Azevedo de Amorim, and A. Sampson, "Modular hardware design with timeline types," *Proceedings of the ACM on Programming Languages (PLDI)*, vol. 7, no. PLDI, pp. 343–367, 2023.

Specific problem: HDL module interfaces expose only port names and bitwidths, so the *temporal* contract—when each input must be valid, when each output appears, how often new inputs are accepted—lives in prose documentation rather than the type signature. Wrapping every module in a valid/ready interface eliminates the problem but imposes overhead that is prohibitive in fine-grained statically scheduled pipelines. Solution: Filament is an HDL with *timeline types*, a PL-style discipline that annotates each port with an availability interval and each module with an *initiation interval* (how often it accepts inputs). A port type `@[T, T+1] in: 32` reads as "a 32-bit value required during cycle T"; a module parameter `<T: 3>` means the module can

start a new computation every three cycles. The type system, inspired by separation logic, treats each instance's timeline as a resource and enforces that uses never overlap; a soundness theorem proves that well-typed Filament programs compile to circuits free of structural hazards. Evaluation asks two questions. Expressivity: Filament assigns types to 14 designs generated by Aetherling (a streaming DSL) and PipelineC (a C-to-hardware compiler), incidentally uncovering five latency bugs in Aetherling's generated interfaces. Efficiency: on a conv2d accelerator, Filament output is competitive with Aetherling's and, when composed with the Reticle low-level FPGA library, uses 7× fewer logic cells. Limitations / future work: Filament only supports statically scheduled pipelines—all interval endpoints must be compile-time constants—so caches, variable-latency arithmetic, and other data-dependent behaviours are out of scope. The type system proves timing safety, not functional correctness of the computation itself. Dynamic-pipeline and polymorphic extensions are identified as future work.

[4] R. Nigam, E. Gabizon, E. Lam, C. Zech, J. Balkind, and A. Sampson, "Parameterized hardware design with latency-abstract interfaces," in *Proceedings of the 31st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2026.

Specific problem: Filament's timeline types are fixed at design time, but real hardware is routinely built from *parameterized generators* (e.g., FloPoCo floating-point cores, Xilinx Vivado IP dividers, Aetherling streams) whose latency depends on input parameters such as bitwidth or target frequency. HDL parameters conventionally flow only from parent to child, so a generated child has no way to propagate "I will have latency L" back to its parent—forcing designers either to hand-rebalance the surrounding pipeline for every parameter choice or to pay the area cost of wrapping the generated module in a latency-insensitive interface. Solution: the authors define *latency-abstract* (LA) interfaces and realize them in Lilac, an HDL extending Filament with *output parameters*—compile-time values emitted by a child module and consumed symbolically by its parent. Lilac intervals can mention output-parameter expressions (e.g., `[G+#L, G+#L+1]` where `#L` is supplied by the child at elaboration time), and an SMT solver discharges the resulting symbolic timing constraints to prove hazard freedom for every valid parameterization. Evaluation compares LA and latency-insensitive (LI) wrapper implementations of a floating-point unit built from FloPoCo-generated adders and multipliers at multiple bitwidths and latencies: the LA design uses 26–33% fewer LUTs and registers and achieves 6.8% higher maximum frequency than the LI baseline—quantifying the hardware cost of the handshake logic that Carloni-style wrapping imposes on fine-grained integration. Limitations / future work: LA interfaces still describe only static pipelines—the output parameter is a compile-time value per elaboration, not a runtime quantity—and the SMT-based check may not scale to deeply nested generator hierarchies. Dynamic latencies remain the domain of Anvil.

[5] P. Pan and C. Batten, "Formal verification of the stall invariant property for latency-insensitive RTL modules," in *Proceedings of the 21st ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE)*, 2023, pp. 148–158.

Specific problem: even when every module uses a valid/ready handshake at its interface, its *internal* logic may contain subtle bugs that surface only under specific

sequences of ingress or egress stall cycles (e.g., a pipeline register enable that forgets to observe a downstream stall). The space of stall sequences grows exponentially in pipeline depth, making simulation-based coverage infeasible. Solution: the authors formalize the *stall invariant property*—the sequence of *informative events* on a module's egress (the cycles on which real data is transferred) must be identical for any two stall patterns—and propose *Bounded Latency Equivalence Checking* (BLEC). BLEC constructs a verification harness containing two copies of the design under verification driven with different stall patterns and uses a commercial formal property-verification tool to prove egress equivalence up to a bounded cycle depth. Evaluation covers three case studies of increasing complexity: a latency-insensitive processing element, a greatest-common-divisor unit, and a pipelined RISC-V processor. In all three cases, BLEC successfully detects injected design bugs that standard simulation misses; counterexample waveforms help localize the root cause. Limitations / future work: BLEC relies on commercial bounded model checkers and requires manual abstractions to avoid state-space explosion on larger designs; scaling to industrial SoCs with millions of state elements is open and likely requires compositional or abstract-interpretation-based reasoning along the very LI interfaces that make the method possible.

[6] J. Z. Yu, A. R. Jha, U. Mathur, T. E. Carlson, and P. Saxena, "Anvil: A general-purpose timing-safe hardware description language," in *Proceedings of the 31st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2026.

Specific problem: timeline types statically track signal lifetimes only when every latency is a compile-time constant, but real hardware is pervasively dynamic—caches hit or miss, page-table walkers take a variable number of cycles per address, dividers stall until done, and bus protocols (e.g., AXI) hand control back to upstream modules at data-dependent events. Wrapping such behaviour in worst-case static contracts defeats the point of caching. Solution: Anvil is a general-purpose timing-safe HDL whose type system indexes time not by integer cycles but by *events*—named, potentially data-dependent points in a per-module event graph. A channel declaration like `right req: (addr @ req->res)` reads "the request address is valid from the `req` event until the next `res` event"; the compiler statically rejects any use of `addr` outside that lifetime. The idea is analogous to Rust's lifetimes lifted from lexical scopes to runtime events. The Anvil compiler (in OCaml) type-checks an event-graph intermediate representation, applies optimization passes, and lowers to SystemVerilog. Evaluation covers ten designs that cannot be expressed in Filament: the CVA6 RISC-V translation-lookaside-buffer (TLB) and page-table walker, OpenTitan's AES cipher core, AXI-Lite demux and mux routers, and three components (FIFO buffer, spill register, passthrough stream FIFO) from the Common Cells library. Versus hand-written SystemVerilog baselines, Anvil output averages 4.5% area and 3.75% power overhead with zero extra clock-cycle latency; on two Filament-style statically scheduled designs (a pipelined ALU and a systolic array) Anvil matches Filament on area. A case-study analysis of six real open-source hardware bugs (OpenTitan, Coyote, ibex, snax-cluster, core2axi) argues that Anvil's type system would have rejected each at compile time. Limitations / future work: Anvil is an early-stage prototype; events must be statically relatable in the event graph (precluding some arbitrary producer–

consumer patterns); integration with large existing SystemVerilog codebases has not yet been validated.